k-NN, Naive Bayes and Support Vector Machines

Lesson 6 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

30 October 2026 · reading time about 80 minutes

Contents

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercise 5 discussed; a problem no line solves	Slides 2–6
0:10–0:30	20	k-nearest neighbours, and choosing k	Slides 7–14
0:30–0:52	22	The curse of dimensionality	Slides 15–21
0:52–1:14	22	**Notebook 01** — k-NN and the curse	Slide 22
1:14–1:26	12	**Break**	Slide 23
1:26–1:48	22	Naive Bayes, and when its assumption holds	Slides 24–34
1:48–2:06	18	**Notebook 02** — where it fails	Slide 35
2:06–2:32	26	Margins, support vectors, the kernel trick	Slides 36–48
2:32–2:50	18	**Notebook 03** — kernels in practice	Slide 49
2:50–3:00	10	The three compared; homework	Slides 50–53
	180	**Total**	**52 slides, 3 notebooks**

---

1. Why this lesson exists

Lessons 3 and 4 built linear models and spent most of their effort on how to fit them honestly. Lesson 5 built the apparatus for telling whether a model works.

None of that told you **which model to reach for**, and this lesson is the first that offers a choice. Three families, each attacking classification from a completely different direction:

- **k-nearest neighbours** makes no assumptions and does no fitting. It remembers the data and votes.
- **Naive Bayes** makes one very strong assumption and, in exchange, trains in a single pass and works in thousands of dimensions.
- **Support vector machines** pick a boundary by a criterion nothing else in this course uses, and then change coordinates when no boundary exists.

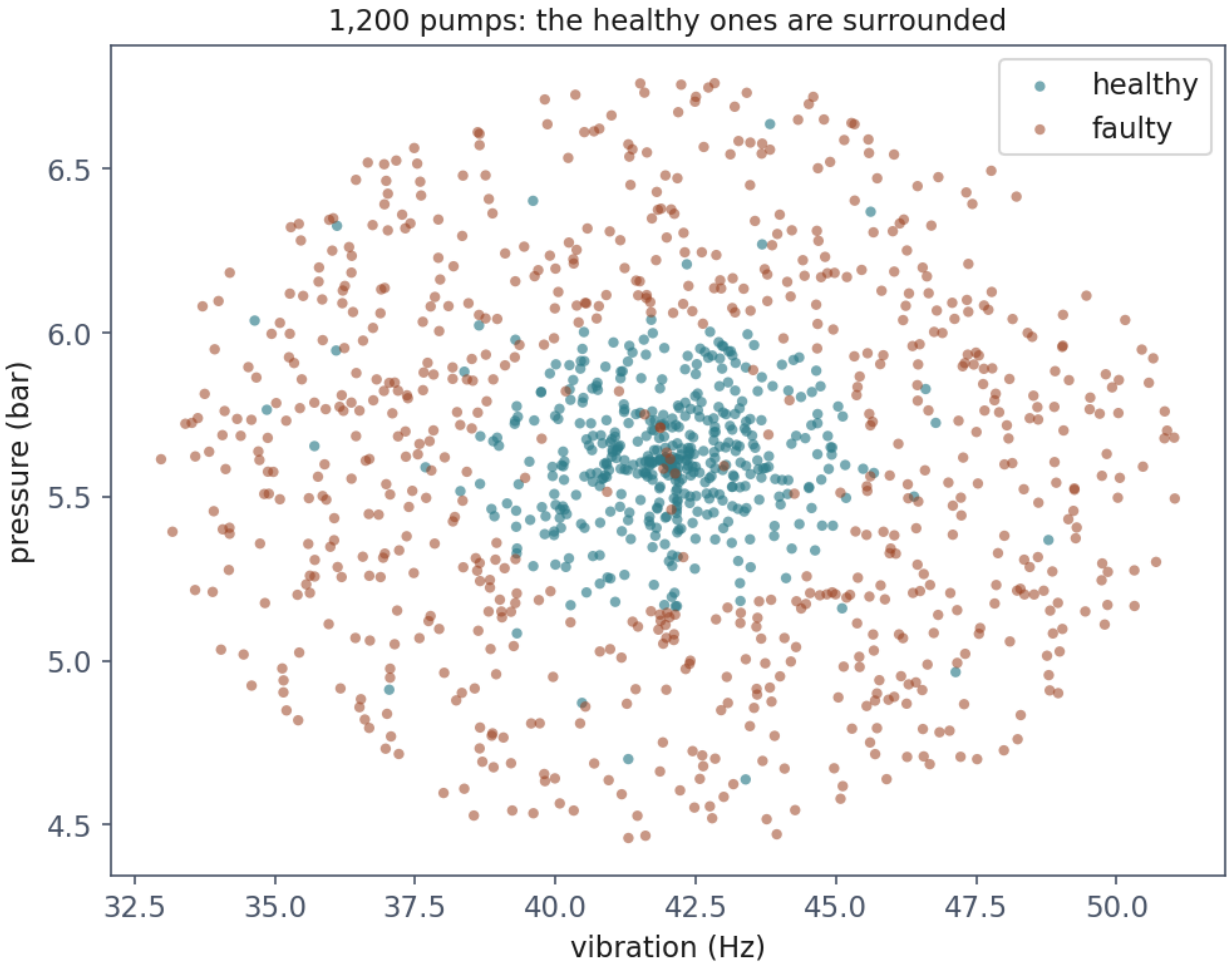
They are not three ways of doing the same thing, and the differences between them are the content of the lesson.

1.1 The dataset, and the point it makes immediately

1,200 industrial pumps, two readings each: vibration in Hz and pressure in bar. A pump has a design operating envelope, and it is **faulty when its readings fall outside that envelope** —

whether too low or too high.

That one physical fact decides everything that follows. The healthy pumps form a disc around the design point; the faulty ones form the annulus around them.



1,200 pumps. The healthy ones are surrounded, because a pump can fail by running too slow as well as too fast. No straight line separates these classes.

4% of the labels are deliberately flipped, so no model can exceed roughly 0.96. That ceiling is lesson 5's noise floor arriving in a classification problem, and it is the number to compare every score below against.

Before building anything, it is worth measuring what a straight boundary costs:

Model	Cross-validated accuracy
Always predict the majority class	0.613
Logistic regression (lesson 4)	**0.613 ± 0.000**

Logistic regression has not been narrowly beaten. It has learned nothing at all, and it predicts "faulty" for every pump because with a straight boundary that is genuinely its best

available answer. The zero standard deviation across folds is the giveaway: a model that gives everything the same answer is perfectly consistent.

---

2. k-nearest neighbours

2.1 The whole algorithm

> To classify a new point, find the k training points closest to it, and take the majority vote.

There is no training step. "Fitting" means storing the data, which is why k-NN is called a **lazy learner** — an unusually honest name for an algorithm.

Two decisions hide in that sentence, and both matter.

What "closest" means. Almost always Euclidean distance:

$$d(x, x') = \sqrt{\sum_{j=1}^{n} (x_j - x'_j)^2}$$

Because every feature contributes to that sum through its own units, the features must be on comparable scales. Vibration ranges over tens of Hz and pressure over a couple of bar, so without scaling vibration would decide every neighbour by itself. Lesson 2's scaling argument arrives here with immediate consequences rather than as hygiene.

What k is. Section 2.2.

2.2 k is the bias-variance dial, made visible

Lesson 5 decomposed error into bias and variance. In k-NN you can watch the trade-off directly by turning one integer.

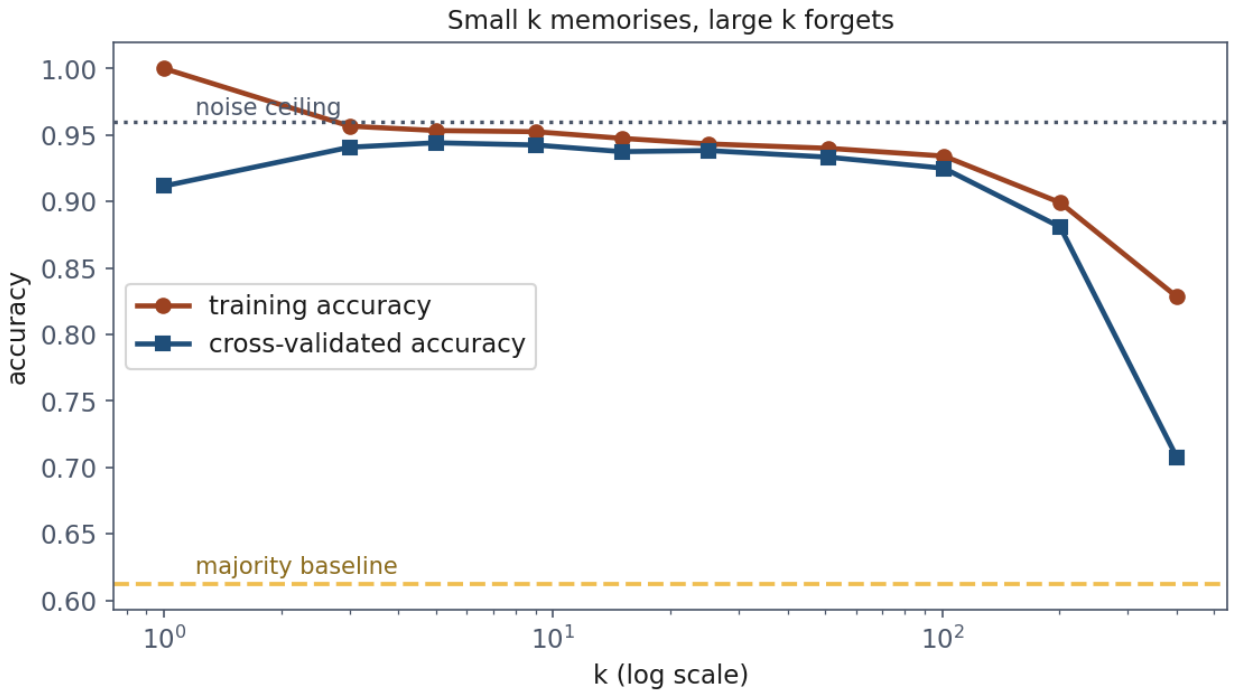
Small k — the boundary follows every point, including mislabelled ones. Low bias, high variance. At $k = 1$ the training accuracy is **exactly 1.000**, always, on any dataset: every point is its own nearest neighbour. That is the purest illustration of lesson 5's point that a training score measures nothing.

Large k — the vote is taken over a wide neighbourhood, so the boundary smooths and eventually stops following real structure. High bias, low variance.

Measured on the pumps:

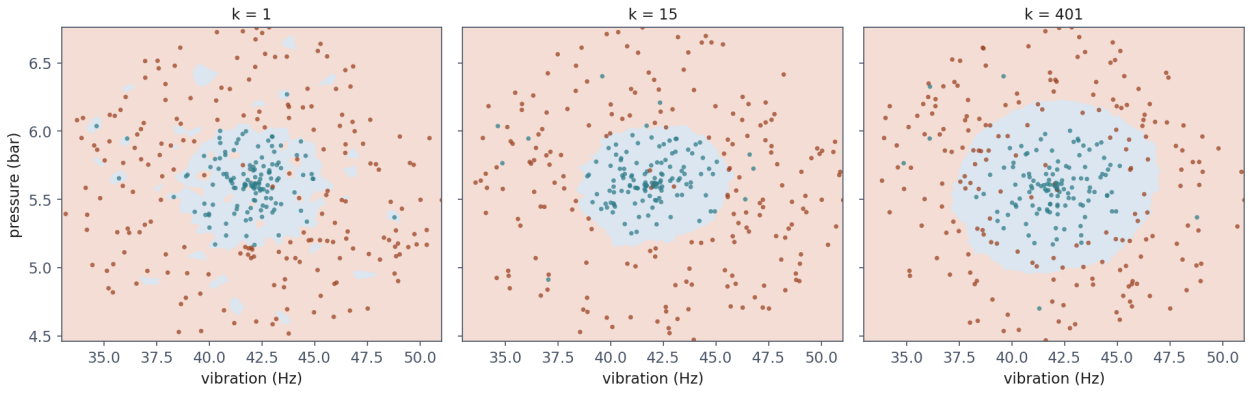
k	Training	Cross-validated
1	**1.000**	0.912
3	0.957	0.941
5	0.953	**0.944**
15	0.948	0.938
51	0.940	0.933
201	0.899	0.881
401	0.828	0.708

k	Training	Cross-validated



The two curves and the two lines that bound them: the noise ceiling at 0.96, which nothing can exceed, and the majority baseline at 0.613, which everything should. Note where the training curve starts.

The best value, 5, sits where the neighbourhood is wide enough to average out the flipped labels and narrow enough to still follow the boundary.



The same data at three values of k. At k = 1 the boundary is ragged, with islands around individual mislabelled points. At k = 15 it is a clean disc, close to the envelope that generated the data. At k = 401 it has ***inflated*** *past the true envelope and swallows faulty pumps — it has not collapsed to one class, it has stopped following the boundary and started averaging over it.*

2.3 What it costs

No training time at all, and you pay at every prediction instead. A naive implementation compares the query against **every** training point:

$$O(mn) \text{ per prediction, for } m \text{ training rows and } n \text{ features}$$

For 1,200 pumps that is nothing. For ten million rows answering a thousand queries a second it is the entire engineering problem, and it is why approximate nearest-neighbour indexes are an industry.

There is a second cost that is easy to miss: **the model is the dataset**. You cannot ship the model without shipping the training data, which is a legal question as much as a practical one — see this lesson's companion reading and lesson 2's.

---

3. The curse of dimensionality

3.1 The demonstration

k-NN just solved a problem that defeated logistic regression completely. Here is the price.

Add columns of **pure noise** — drawn from a normal distribution, unrelated to anything. The two real readings are untouched, so the problem is exactly as solvable as before. Only the number of columns changes.

Noise columns	Total columns	Accuracy	Above baseline
0	2	0.938	+0.325
5	7	0.854	+0.242
10	12	0.762	+0.149
25	27	0.662	+0.049
50	52	0.602	**−0.011**
100	102	0.578	−0.035

The signal never left. Those two columns are still there and still sufficient. But by fifty noise columns, k-NN is below the majority baseline: a model that ignored the data entirely would now do better.

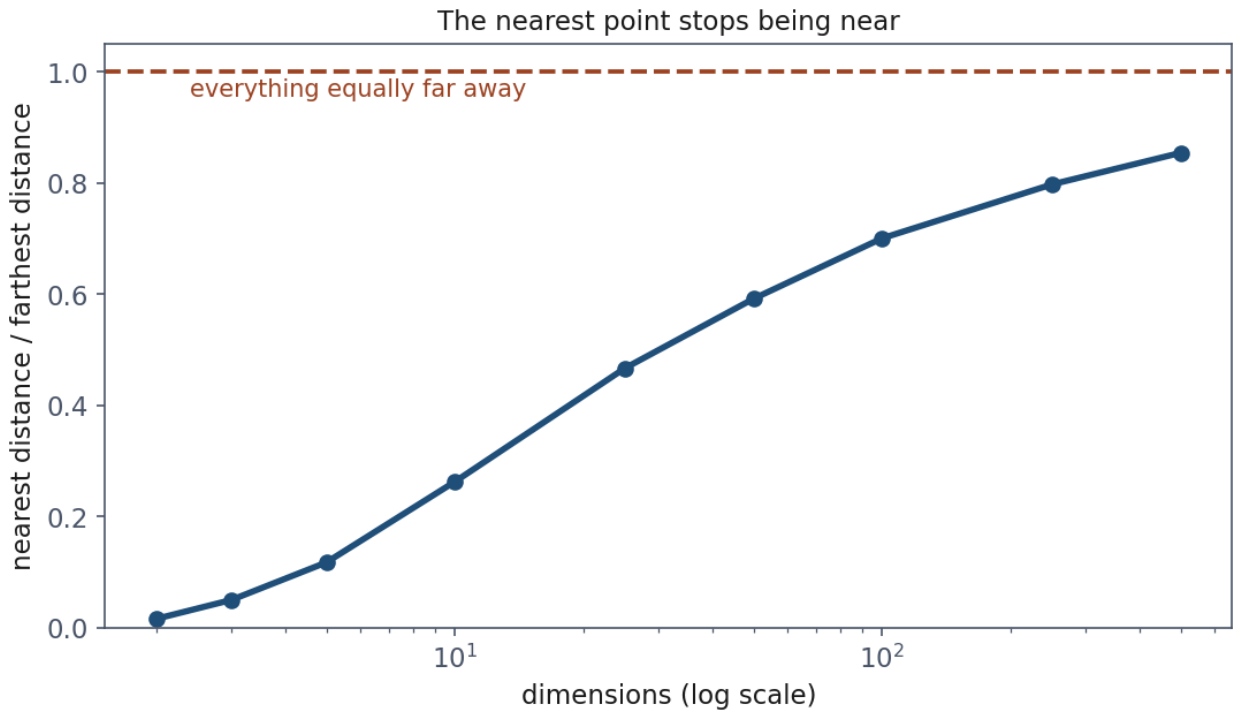
3.2 Why: distances stop varying

The mechanism is geometry, not statistics, and it has nothing to do with k-NN specifically.

The picture first. Scatter points uniformly in a cube, pick one, and measure the distance to its nearest neighbour and to its farthest. In two dimensions those are very different numbers, and that difference is exactly what makes "nearest" a meaningful word.

Now add dimensions. Each new dimension contributes its own squared difference to every distance. Those contributions average out, and as the number of terms grows, all the distances converge on the same value.

Dimensions	nearest ÷ farthest
2	0.016
10	0.263
50	0.592
100	**0.701**
500	0.855



As dimensions grow, the nearest point stops being near. The ratio climbs towards 1, where every point is the same distance from every other.

In two dimensions the nearest point is about 2% as far away as the farthest. "Nearest" is a strong claim.

In one hundred dimensions it is 70% as far away as the farthest. The nearest point is barely nearer than a random one, and a vote among "the five nearest" is close to a vote among five taken at random.

3.3 What the curse is, and is not

It is **not** that high-dimensional problems are inherently unlearnable. Lesson 9's networks work in thousands of dimensions.

It is that **methods built on distance lose their footing**, because the quantity they depend on stops varying. That includes k-NN, k-means (lesson 8), and RBF kernels — which is why the RBF SVM also degraded in the table above, though more slowly.

The predictable mistake, and why the instinct is sound. Adding features because they might help is good practice with a linear model, where an irrelevant feature costs you a coefficient

near zero and very little else. With k-NN it costs you a dimension in the distance, and dimensions are what the method is made of. The same habit, transferred one lesson later, does real damage.

---

4. Naive Bayes

4.1 Bayes' rule, and where the difficulty is

We want $P(y = c \mid x)$. Bayes' rule turns it into quantities we can estimate:

$$P(y = c \mid x) = \frac{P(x \mid y = c)\, P(y = c)}{P(x)}$$

$P(y = c)$ is the prior — a count. $P(x)$ is identical across classes, so it cannot change which class wins and can be dropped. Everything hard is in $P(x \mid y = c)$: **the probability of this exact combination of readings among examples of that class.**

With two features that is a two-dimensional density and we could estimate it. With twenty it is a twenty-dimensional one, and no quantity of data populates a twenty-dimensional space — the curse of Section 3, arriving from an entirely different direction.

4.2 The assumption

Given the class, the features are independent of one another.

If that holds, the joint density factorises:

$$P(x \mid y = c) = \prod_{j=1}^{n} P(x_j \mid y = c)$$

and each factor is a one-dimensional density estimated from the rows of that class. The classifier is then

$$\hat{y} = \arg\max_{c}\ P(y = c) \prod_{j=1}^{n} P(x_j \mid y = c)$$

In practice this is computed as a sum of logarithms, for the same underflow reason as lesson 4's log-likelihood: a product of hundreds of small probabilities is zero in floating point.

$$\hat{y} = \arg\max_{c} \left[\log P(y = c) + \sum_{j=1}^{n} \log P(x_j \mid y = c) \right]$$

Why it is such a good bargain. One n-dimensional estimation problem becomes n one-dimensional ones. Training is a single pass computing means and variances. It needs very little data per feature, and adding features costs almost nothing.

Why it is almost never true. Vibration and pressure are both driven by the operating point. "New" is not independent of "York" given the topic. Symptoms co-occur.

The interesting question is therefore not whether the assumption holds, but **when being wrong about it costs you nothing**.

4.3 On the pumps, the assumption happens to hold

Model	Accuracy
Majority baseline	0.613
Logistic regression	0.613
Gaussian Naive Bayes	**0.933**
k-NN, $k = 5$	0.944

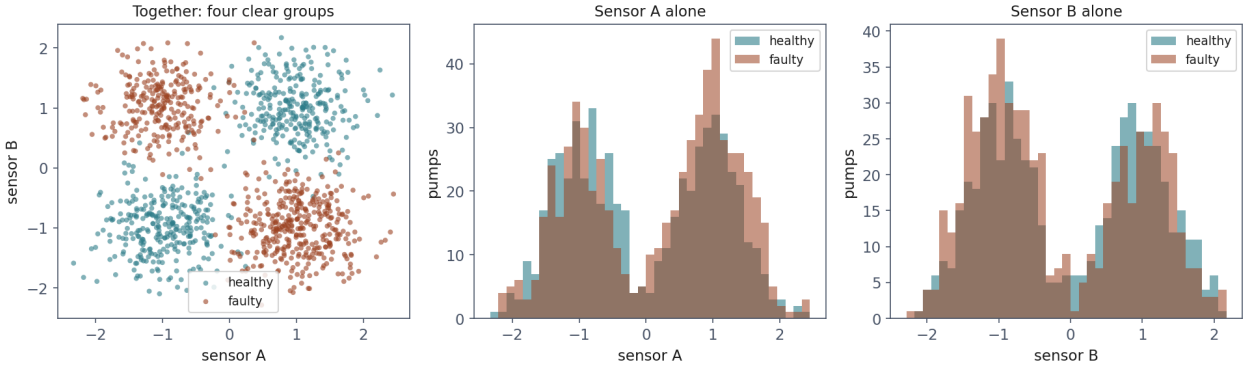
A point behind k-NN, and an enormous distance ahead of the linear model. So test the assumption directly — remembering that it concerns independence *given the class*, which is not the same as independence overall:

Correlation between the two readings	
overall	−0.046
within healthy pumps	**−0.006**
within faulty pumps	**−0.049**

Within each class the readings are essentially uncorrelated. Naive Bayes is doing well here because its assumption is true here — which is far more useful to know than the score, because it tells you when to expect the score to hold.

4.4 One step away, worse than guessing

Now a second pair of sensors on the same fleet. The pump is faulty when **exactly one** of the two readings is high: both high is the designed high-load mode, both low is idle, and one without the other is a mismatch between demand and delivery.



Left: together, the two sensors show four clear groups and a perfectly learnable rule. Middle and right: what Naive Bayes gets to see — each sensor alone, where the two classes sit almost exactly on top of one another.

Model	Accuracy
Majority baseline	0.523
Gaussian Naive Bayes	**0.404**
Logistic regression	0.393
SVM, linear kernel	0.606
k-NN, $k = 5$	0.967
SVM, RBF kernel	0.972

0.404 — below chance, and well below the majority baseline.

Being below chance looks impossible, and the explanation is worth having. The class means on each sensor differ by about 0.17 against a spread near 1, purely as an artefact of a finite sample. That accident is the *only* per-feature evidence available, Naive Bayes has nothing else to multiply, and in this sample it points the wrong way.

A model with no signal does not sit politely at 50%. It follows whatever spurious structure it can find.

4.5 Its probabilities are not probabilities

Measured on the interacting data: mean confidence when correct **0.567**, mean confidence when wrong **0.555**. It cannot tell the difference.

The more common complaint runs the other way, and it is not measured on this dataset — it needs correlated features, which the pump data does not have. When features *are* correlated, multiplying their probabilities counts the same evidence repeatedly, and Naive Bayes becomes wildly overconfident. Ten near-copies of one informative column is enough: accuracy around 0.75, and two thirds of the predictions asserted above 0.999. Try it — it takes four lines.

Either way, lesson 5's distinction applies: **the ranking may be useful while the probabilities are not.** Naive Bayes is the classic example of that gap, and the reason it should not be used where a calibrated probability is needed — such as the cost calculation of lesson 4, Section 7.2.

4.6 When to use it anyway

Very high dimensions with little data. Text classification is canonical: tens of thousands of word-count features, an assumption that is transparently false, and a method that works anyway — because being roughly right in ten thousand dimensions beats being unable to estimate anything at all.

As a baseline. It trains in one pass. If your tuned model does not beat Naive Bayes, you have learned something quickly and cheaply.

Not when the signal is an interaction. No quantity of data repairs Section 4.4, because the model cannot represent what you are asking of it.

---

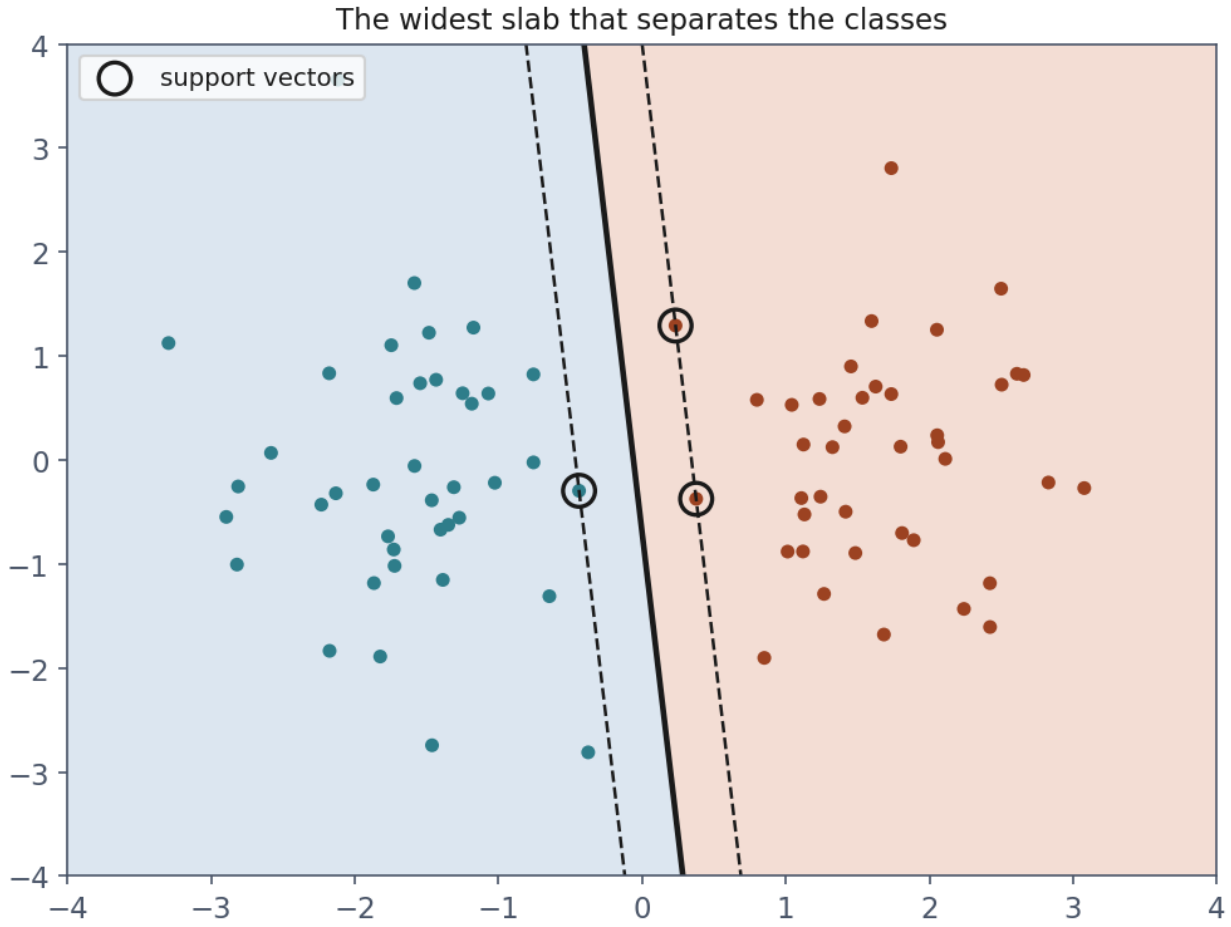
5. Support vector machines

5.1 The margin: a different criterion

On separable data there are infinitely many separating lines, and **every one of them has zero training error**. Minimising the error therefore cannot choose between them; logistic regression breaks the tie with log loss, which is one answer among several.

The support vector machine uses a different one: choose the boundary with the **widest margin**. Push a slab out from the boundary until it touches the nearest point of each class, and pick the boundary that makes the slab thickest.

The intuition. A boundary passing close to a training point is one small perturbation away from getting it wrong. Maximising the distance to the closest points chooses the boundary that tolerates the most movement in the data before it changes its mind. That is a statement about generalisation, not about fit.



The solid line is the boundary, the dashed lines the edges of the slab, and the circled points the support vectors touching it. Of eighty points, three determine the answer; move any of the others and nothing changes.

5.2 The optimisation, and the soft margin

For labels written as $y_i \in \{-1, +1\}$, the margin of a separating hyperplane $w^\top x + b$ is $2/\|w\|$, so maximising it means minimising $\|w\|$ subject to every point being correctly outside the slab:

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i\,(w^\top x_i + b) \geq 1 \ \ \forall i$$

Real data is not separable — ours has 4% of its labels flipped — and this problem then has no solution at all. The fix is to allow violations ξ_i and charge for them:

$$\min_{w,b,\xi} \ \tfrac{1}{2}\|w\|^2 + C\sum_{i=1}^{m} \xi_i \quad \text{subject to} \quad y_i\,(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

C is the price of a training error. Large C makes violations expensive, so the model contorts to classify everything: narrow margin, low bias, high variance. Small C buys a wider, calmer boundary at the cost of some errors.

It is the same dial as k in Section 2 and λ in lesson 3, in a third costume — and note the direction, which catches people out: **large C means less regularisation.**

5.3 On the pumps, the margin alone does not help

Model	Accuracy	Support vectors
SVM, linear kernel	**0.613 ± 0.000**	947 of 1,200 (**79%**)
SVM, RBF kernel	0.947 ± 0.005	278 of 1,200 (23%)

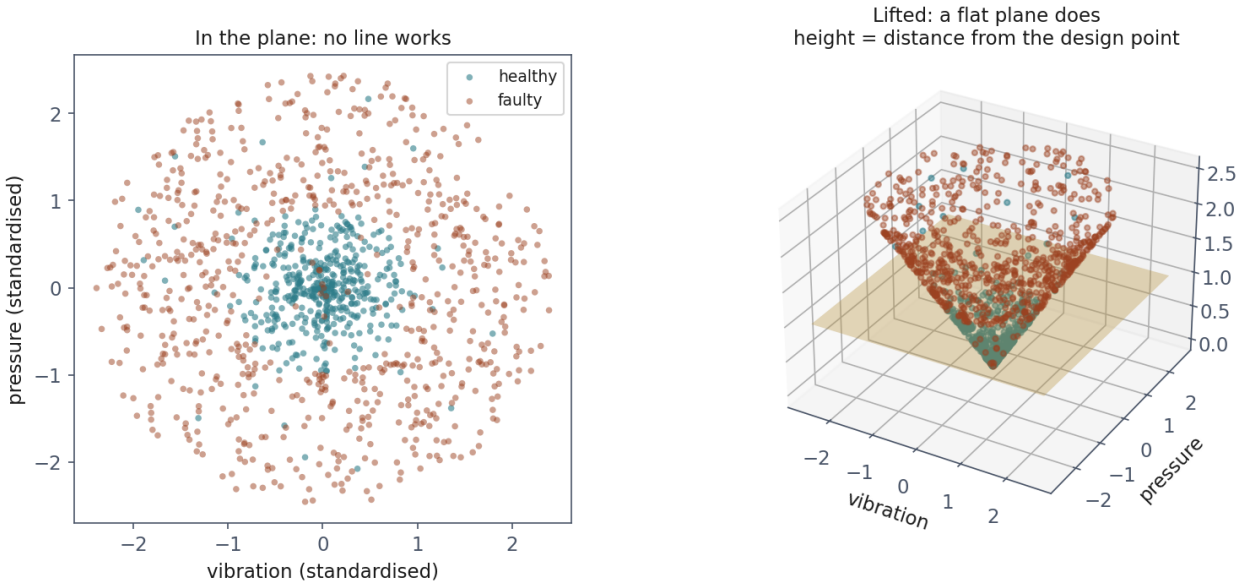
The linear kernel scores the base rate, exactly as logistic regression did. Choosing the best straight line does not help when no straight line works.

The support-vector counts say the same thing in another language. The linear model needs 79% of the training set, because almost every point sits on or inside the margin — there is no slab that separates anything. **A high support-vector fraction is a free warning** that the model is struggling to find room.

5.4 The kernel trick

The picture first. Our healthy pumps sit in a disc, the faulty ones around it, and no line separates them *in the plane*. Add a third coordinate — the distance from the centre — and lift each point to that height. Healthy pumps rise a little, faulty ones a lot, and a flat horizontal plane separates them perfectly.

The classes were always separable. They needed different coordinates.



The same 1,200 pumps twice: in the plane where they were measured, and lifted by their distance from the design point. The gold plane does what no line could.

That is the idea in general: map into a space $\phi(x)$ where a linear boundary works, and run the linear method there. The obstacle is that useful spaces are enormous, sometimes infinite-dimensional, and computing $\phi(x)$ would be impossible.

The trick is that the SVM never needs $\phi(x)$. Its solution depends on the data only through inner products between pairs of points, and for the right maps there is a function that returns the inner product *in the new space* while computing only with the original one:

$$K(x, x') = \langle \phi(x), \phi(x') \rangle$$

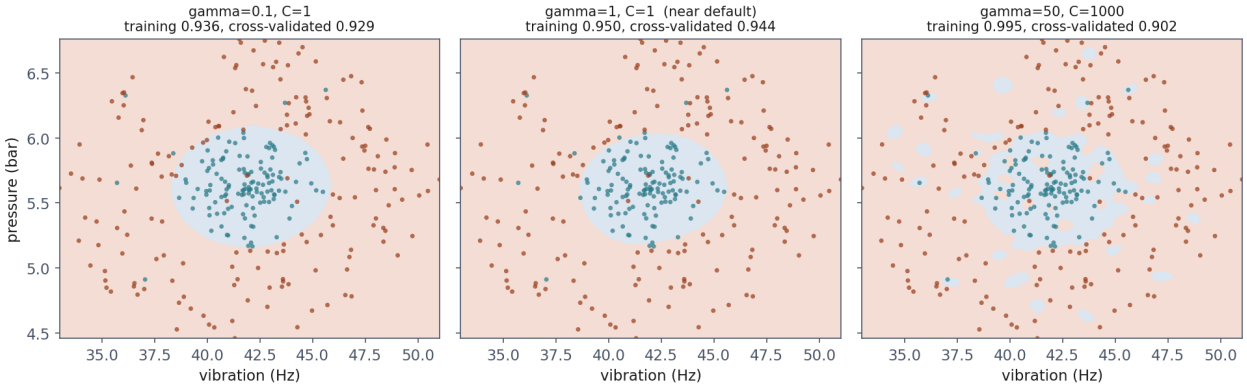
The standard choice, and scikit-learn's default, is the **radial basis function**:

$$K(x, x') = \exp\left(-\gamma \|x - x'\|^2\right)$$

which corresponds to an infinite-dimensional feature space and costs one exponential per pair. We chose the lift in the picture above by knowing the answer; the RBF kernel does something equivalent without being told, which is why it works where nobody could guess the right coordinates.

5.5 Gamma, C, and overfitting you can see

γ sets how far a single training point's influence reaches. Small γ: wide reach, smooth boundary. Large γ: each point influences only its immediate neighbourhood, and the boundary can dissolve into islands.



Three settings, with the training and cross-validated scores in each title. Read them together, as lesson 5 taught.

Setting	Training	Cross-validated
$\gamma = 0.1$, $C = 1$	0.936	0.929
$\gamma = 1$, $C = 1$	0.950	**0.944**
$\gamma = 50$, $C = 1000$	**0.995**	0.902

The last row has the **highest training score and the lowest honest one** — the signature of overfitting, and here it is visible as well as measurable: the boundary has broken into bubbles around individual points, including the mislabelled ones.

Choosing between these three on the training score would select the worst model with complete confidence.

---

6. The three compared

Everything in this lesson on the same 1,200 pumps, cross-validated:

Model	Accuracy
Majority baseline	0.613
Logistic regression (lesson 4)	0.613
SVM, linear kernel	0.613
Gaussian Naive Bayes	0.933
k-NN, $k = 5$	0.944
SVM, RBF kernel	**0.947**
noise ceiling	*0.96*

Three methods reach the ceiling by three unrelated routes — remembering the neighbourhood, assuming independence, bending the space — and two do not, both of them linear.

The gap between best and worst is 0.334, larger than any difference this course has shown between a good model and a tuned one. That is the lesson of the table: **choosing the right**

family matters far more than tuning the wrong one, and the way to tell which family you need is to look at the data first, as lesson 2 insisted.

How to choose, in practice

Situation	Reach for
Few features, plenty of data, odd-shaped boundary	k-NN, or an RBF SVM
Very many features, little data	Naive Bayes
Need a calibrated probability	Not Naive Bayes
Need the model to be small, or fast at prediction	SVM, not k-NN
Signal is an interaction between features	k-NN or a kernel; not Naive Bayes
Need to explain the decision to the person affected	Neither — lesson 3's Resources

---

7. Summary

- **No straight line separates the pumps**, and both linear models score exactly the base rate, 0.613, with zero variance across folds.
- **k-NN learns nothing and scores 0.944.** Scale first; distance is all it has.
- k **is the bias-variance dial**: training accuracy is exactly 1.000 at $k = 1$ and means nothing.
- **The curse of dimensionality is about distance, not difficulty.** Fifty noise columns took k-NN below the baseline. In 100 dimensions the nearest point is **70%** as far away as the farthest.
- **Naive Bayes assumes independence given the class.** On the pumps that holds (-0.006 within class) and it scores 0.933; where the signal is an interaction it scores **0.404**, below chance.
- **Its probabilities are not calibrated**, in either direction.
- **The margin is a criterion distinct from the error**, and support vectors are the model. A high support-vector fraction is a free warning.
- **The kernel trick separates by changing coordinates**, through inner products in a space never computed.
- **Choosing the family beats tuning the wrong one**: 0.613 to 0.947 on identical data.

Homework

`Exercises/06_knn_naive_bayes_svm.md`, discussed at the start of Lesson 7, **Friday 6 November 2026**.

Notation used in this lesson

Symbol	Meaning
x, x'	two feature vectors
$d(x, x')$	Euclidean distance between them

Symbol	Meaning
k	number of neighbours voting
m, n	number of examples, number of features
w, b	the hyperplane's coefficients and intercept
ξ_i	how far example i violates the margin
C	the price of a margin violation
γ	how far one point's influence reaches, in an RBF kernel
ϕ	the map into the higher-dimensional space
K	the kernel, an inner product in that space